



Factory Method y Builder aplicados al Sistema de gestión de tutorías

Estudiante: Joffre Barre Veliz

Repositorio GitHub: https://github.com/joffrebarrev/Sistema_gestion_tutorias

1. Propósito

Implementar y comparar Factory Method y Builder sobre problemas concretos del Sistema de gestión de tutorías, justificando técnicamente qué problema resuelve cada patrón, cómo se representa en UML y cómo se traduce a Java.

2. Caso base

El Sistema de gestión de tutorías requiere diferentes mecanismos de notificación (Email, Push, SMS y, como variante adicional, Teams) y una Reserva cuya configuración incluye múltiples datos obligatorios y opcionales.

Parte A | Factory Method

A.1 Problema inicial

Una primera implementación concentraría la decisión del tipo de notificador en una cadena de condicionales:

```
public Notificador crear(String tipo) {  
    if (tipo.equals("EMAIL")) return new NotificadorEmail();  
    if (tipo.equals("PUSH")) return new NotificadorPush();  
    if (tipo.equals("SMS")) return new NotificadorSms();  
    throw new IllegalArgumentException("Tipo no soportado");  
}
```

El problema no es el uso de if, sino que cada nuevo canal obliga a modificar este mismo método y dispersa el conocimiento de las clases concretas por todo el código que lo invoque.

A.2 Implementación completa

Notificador.java

```
package edu.uees.patrones.factory;  
  
public interface Notificador {  
  
    void enviar(String destino, String mensaje);  
}
```

NotificadorEmail.java

```
package edu.uees.patrones.factory;  
  
public class NotificadorEmail implements Notificador {  
  
    @Override  
    public void enviar(String destino, String mensaje) {  
        System.out.println("[EMAIL]");  
        System.out.println("Destino: " + destino);  
        System.out.println("Mensaje: " + mensaje);  
    }  
}
```



```
}
```

NotificadorPush.java

```
package edu.uees.patrones.factory;

public class NotificadorPush implements Notificador {

    @Override
    public void enviar(String destino, String mensaje) {
        System.out.println("[PUSH]");
        System.out.println("Usuario: " + destino);
        System.out.println("Mensaje: " + mensaje);
    }
}
```

NotificadorSms.java

```
package edu.uees.patrones.factory;

public class NotificadorSms implements Notificador {

    @Override
    public void enviar(String destino, String mensaje) {
        System.out.println("[SMS]");
        System.out.println("Telefono: " + destino);
        System.out.println("Mensaje: " + mensaje);
    }
}
```

NotificadorTeams.java (variante de extensibilidad)

```
package edu.uees.patrones.factory;

public class NotificadorTeams implements Notificador {

    @Override
    public void enviar(String destino, String mensaje) {
        System.out.println("[TEAMS]");
        System.out.println("Usuario: " + destino);
        System.out.println("Mensaje: " + mensaje);
    }
}
```

NotificadorCreator.java

```
package edu.uees.patrones.factory;

public abstract class NotificadorCreator {

    protected abstract Notificador crearNotificador();

    public void notificar(String destino, String mensaje) {
        Notificador notificador = crearNotificador();
        notificador.enviar(destino, mensaje);
    }
}
```

EmailCreator.java / PushCreator.java / SmsCreator.java / TeamsCreator.java

```
public class EmailCreator extends NotificadorCreator {
```



```
@Override
protected Notificador crearNotificador() {
    return new NotificadorEmail();
}

public class PushCreator extends NotificadorCreator {
    @Override
    protected Notificador crearNotificador() {
        return new NotificadorPush();
    }
}

public class SmsCreator extends NotificadorCreator {
    @Override
    protected Notificador crearNotificador() {
        return new NotificadorSms();
    }
}

public class TeamsCreator extends NotificadorCreator {
    @Override
    protected Notificador crearNotificador() {
        return new NotificadorTeams();
    }
}
```

AppFactory.java

```
package edu.uees.patrones.factory;

public class AppFactory {

    public static void main(String[] args) {
        System.out.println("=== FACTORY METHOD | UEES ===");

        NotificadorCreator emailCreator = new EmailCreator();
        emailCreator.notificar("estudiante@uees.edu.ec", "Su tutoria fue confirmada");

        System.out.println();
        NotificadorCreator pushCreator = new PushCreator();
        pushCreator.notificar("usuario-001", "Tiene una tutoria en 30 minutos");

        System.out.println();
        NotificadorCreator smsCreator = new SmsCreator();
        smsCreator.notificar("+593999999999", "Su tutoria fue confirmada");

        System.out.println();
        NotificadorCreator teamsCreator = new TeamsCreator();
        teamsCreator.notificar("usuario.teams@uees.edu.ec", "Su tutoria fue confirmada");
    }
}
```

A.3 Demostración de creación y uso

Salida esperada al ejecutar AppFactory (mvn exec:java -Dexec.mainClass="edu.uees.patrones.factory.AppFactory"):

```
=== FACTORY METHOD | UEES ===
```



```
[EMAIL]
Destino: estudiante@uees.edu.ec
Mensaje: Su tutoría fue confirmada

[PUSH]
Usuario: usuario-001
Mensaje: Tiene una tutoría en 30 minutos

[SMS]
Telefono: +593999999999
Mensaje: Su tutoría fue confirmada

[TEAMS]
Usuario: usuario.teams@uees.edu.ec
Mensaje: Su tutoría fue confirmada
```

A.4 Variante adicional (extensibilidad)

Se incorporó NotificadorTeams y TeamsCreator como cuarta variante para evidenciar extensibilidad sin modificar el código existente.

A.5 Qué clases cambian y cuáles permanecen estables

- Permanecen estables: Notificador (contrato), NotificadorCreator (declara el Factory Method y el flujo notificar()), y todas las clases concretas ya existentes (NotificadorEmail, NotificadorPush, NotificadorSms, EmailCreator, PushCreator, SmsCreator).
- Cambian / se agregan: únicamente dos archivos nuevos por cada variante — un ConcreteProduct (ej. NotificadorTeams) y un ConcreteCreator (ej. TeamsCreator). Ningún archivo existente requiere modificación.

Esto confirma que Factory Method aísla correctamente la variabilidad del tipo concreto y respeta el principio Abierto/Cerrado.

Parte B | Builder

B.1 Problema del constructor inicial

Un constructor con diez parámetros posicionales es difícil de leer y propenso a errores:

```
Reserva reserva = new Reserva(
    "Ana Torres", "Carlos Perez", fechaHora, Modalidad.VIRTUAL,
    "Revisión de proyecto", "Analizar UML", Prioridad.ALTA,
    true, "https://meet.example/tutoria", 45
);
```

B.2 Campos obligatorios y opcionales

- Obligatorios: estudiante, docente, fechaHora, modalidad.
- Opcionales con valor por defecto: motivo ("Sin motivo especificado"), observacion (""), prioridad (MEDIA), recordatorio (false), enlace (""), duracionMinutos (30, validado > 0).



B.3 Implementación completa

Modalidad.java / Prioridad.java

```
public enum Modalidad { PRESENCIAL, VIRTUAL }

public enum Prioridad { BAJA, MEDIA, ALTA }
```

Reserva.java (Product immutable)

```
package edu.uees.patrones.builder;

import java.time.LocalDateTime;

public final class Reserva {

    private final String estudiante;
    private final String docente;
    private final LocalDateTime fechaHora;
    private final Modalidad modalidad;
    private final String motivo;
    private final String observacion;
    private final Prioridad prioridad;
    private final boolean recordatorio;
    private final String enlace;
    private final int duracionMinutos;

    Reserva(String estudiante, String docente, LocalDateTime fechaHora, Modalidad modalidad,
            String motivo, String observacion, Prioridad prioridad, boolean recordatorio,
            String enlace, int duracionMinutos) {
        this.estudiante = estudiante;
        this.docente = docente;
        this.fechaHora = fechaHora;
        this.modalidad = modalidad;
        this.motivo = motivo;
        this.observacion = observacion;
        this.prioridad = prioridad;
        this.recordatorio = recordatorio;
        this.enlace = enlace;
        this.duracionMinutos = duracionMinutos;
    }

    // getters omitidos por espacio – ver repositorio GitHub
}
```

ReservaBuilder.java (Fluent API + validación)

```
package edu.uees.patrones.builder;

import java.time.LocalDateTime;

public class ReservaBuilder {

    private String estudiante;
    private String docente;
    private LocalDateTime fechaHora;
    private Modalidad modalidad;

    private String motivo = "Sin motivo especificado";
```



```
private String observacion = "";
private Prioridad prioridad = Prioridad.MEDIA;
private boolean recordatorio = false;
private String enlace = "";
private int duracionMinutos = 30;

public ReservaBuilder estudiante(String v) { this.estudiante = v; return this; }
public ReservaBuilder docente(String v) { this.docente = v; return this; }
public ReservaBuilder fechaHora(LocalDateTime v) { this.fechaHora = v; return this; }
public ReservaBuilder modalidad(Modalidad v) { this.modalidad = v; return this; }
public ReservaBuilder motivo(String v) { this.motivo = v; return this; }
public ReservaBuilder observacion(String v) { this.observacion = v; return this; }
public ReservaBuilder prioridad(Prioridad v) { this.prioridad = v; return this; }
public ReservaBuilder recordatorio(boolean v) { this.recordatorio = v; return this; }
public ReservaBuilder enlace(String v) { this.enlace = v; return this; }
public ReservaBuilder duracionMinutos(int v) { this.duracionMinutos = v; return this; }

public Reserva build() {
    validar();
    return new Reserva(estudiante, docente, fechaHora, modalidad, motivo,
        observacion, prioridad, recordatorio, enlace, duracionMinutos);
}

private void validar() {
    if (estudiante == null || estudiante.isBlank())
        throw new IllegalStateException("El estudiante es obligatorio");
    if (docente == null || docente.isBlank())
        throw new IllegalStateException("El docente es obligatorio");
    if (fechaHora == null)
        throw new IllegalStateException("La fecha y hora son obligatorias");
    if (modalidad == null)
        throw new IllegalStateException("La modalidad es obligatoria");
    if (duracionMinutos <= 0)
        throw new IllegalStateException("La duracion debe ser mayor que cero");
}
}
```

AppBuilder.java (dos configuraciones + validación)

```
package edu.uees.patrones.builder;

import java.time.LocalDateTime;

public class AppBuilder {

    public static void main(String[] args) {
        System.out.println("=== BUILDER | UEES ===");

        Reserva reservaVirtual = new ReservaBuilder()
            .estudiante("Ana Torres")
            .docente("Carlos Perez")
            .fechaHora(LocalDateTime.of(2026, 8, 29, 18, 0))
            .modalidad(Modalidad.VIRTUAL)
            .motivo("Revision del proyecto")
            .observacion("Analizar diagrama UML")
            .prioridad(Prioridad.ALTA)
            .recordatorio(true)
            .enlace("https://meet.example/tutoria")
            .duracionMinutos(45)
    }
}
```



```
.build();

System.out.println("Reserva 1 (configuracion completa:");
System.out.println(reservaVirtual);

Reserva reservaPresencial = new ReservaBuilder()
    .estudiante("Maria Lopez")
    .docente("Juan Garcia")
    .fechaHora(LocalDateTime.of(2026, 8, 30, 10, 0))
    .modalidad(Modalidad.PRESENCIAL)
    .build();

System.out.println("Reserva 2 (valores por defecto:");
System.out.println(reservaPresencial);

try {
    Reserva invalida = new ReservaBuilder()
        .docente("Carlos Perez")
        .fechaHora(LocalDateTime.now())
        .modalidad(Modalidad.VIRTUAL)
        .build();
    System.out.println(invalida);
} catch (IllegalStateException e) {
    System.out.println("Validacion correcta: " + e.getMessage());
}
}
```

B.4 Dos configuraciones y validación demostrada

Salida esperada al ejecutar AppBundle (mvn exec:java -Dexec.mainClass="edu.uees.patrones.builder.AppBuilder"):

```
=== BUILDER | UEES ===
Reserva 1 (configuracion completa):
Reserva{estudiante='Ana Torres', docente='Carlos Perez', fechaHora=2026-08-29T18:00,
modalidad=VIRTUAL, motivo='Revision del proyecto', observacion='Analizar diagrama UML',
prioridad=ALTA, recordatorio=true, enlace='https://meet.example/tutoria',
duracionMinutos=45}

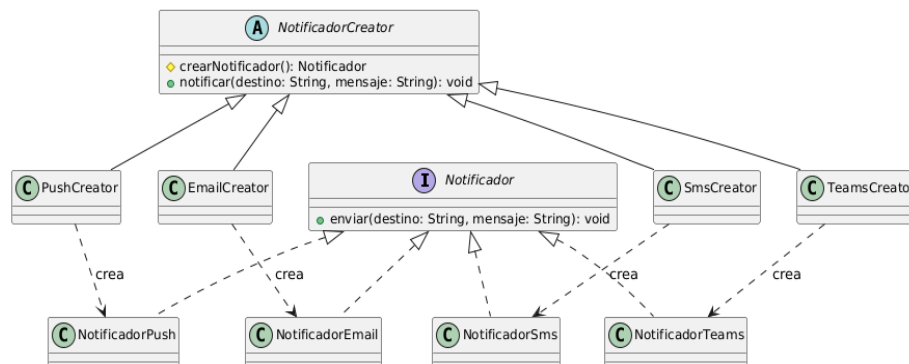
Reserva 2 (valores por defecto):
Reserva{estudiante='Maria Lopez', docente='Juan Garcia', fechaHora=2026-08-30T10:00,
modalidad=PRESENCIAL, motivo='Sin motivo especificado', observacion='', prioridad=MEDIA,
recordatorio=false, enlace='', duracionMinutos=30}

Validacion correcta: El estudiante es obligatorio
```

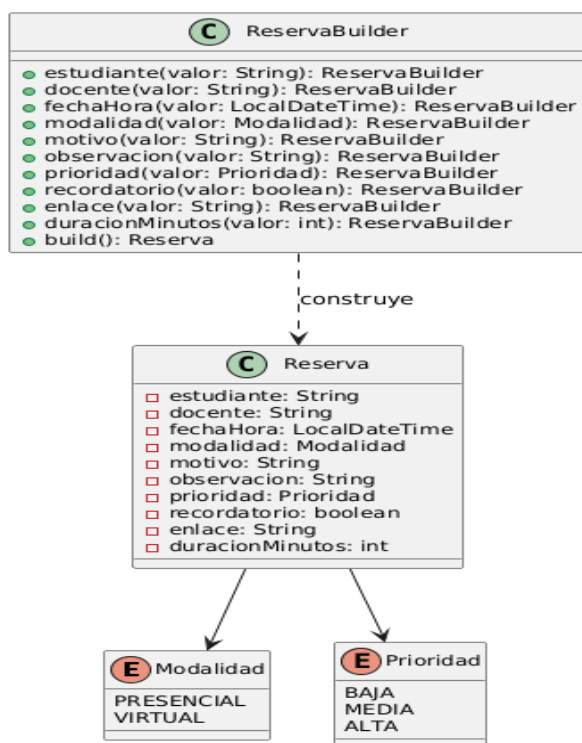


Parte C | Diagramas UML

C.1 Factory Method



C.2 Builder



Ambos diagramas corresponden exactamente a las clases implementadas en el repositorio (docs/factory-method.puml y docs/builder.puml), cumpliendo el requisito de coherencia entre UML y código Java.

Parte D | Comparación técnica

Criterio	Factory Method	Builder
Problema que resuelve	Crear un objeto sin que el cliente conozca ni	Ensamblar progresivamente un objeto con



	decida la clase concreta que se instancia.	muchos campos obligatorios y opcionales.
Variabilidad principal	El tipo concreto de producto (Email, Push, SMS, Teams).	La combinación de configuraciones opcionales de un mismo tipo (Reserva).
Participantes	Product, ConcreteProduct, Creator, ConcreteCreator.	Product (Reserva) y Builder (ReservaBuilder) con Fluent API.
Ventaja principal	Nuevas variantes se agregan sin modificar código existente (OCP).	Construcción legible, con nombres explícitos por configuración.
Costo / consecuencia	Aumenta el número de clases; introduce indirección.	Clase adicional; posible duplicación de campos con el Product.
Cuándo utilizarlo	Cuando el tipo concreto a crear varía y se prevén nuevas variantes.	Cuando existen varios campos opcionales y combinaciones válidas distintas.
Cuándo evitarlo	Si solo existe un tipo concreto y no se prevén variantes.	Si el objeto tiene pocos campos, todos obligatorios y sin ambigüedad.

Parte E | Git y GitHub

El repositorio se construyó con la siguiente evolución progresiva de commits (ver Sección 14 de este documento para los comandos exactos ejecutados desde la terminal):

```
feat: crear contrato de notificacion
feat: implementar factory method
feat: agregar nueva variante de notificacion
feat: implementar reserva builder
docs: agregar diagramas UML
docs: documentar comparacion de patrones
```

Conclusiones

Factory Method y Builder resuelven problemas de creación de objetos distintos y no intercambiables. Factory Method aísla la variabilidad de qué tipo concreto instanciar, permitiendo agregar nuevos canales de notificación sin modificar código existente, tal como se demostró al incorporar Teams sin tocar Email, Push ni Sms.

Builder, en cambio, resuelve cómo ensamblar progresivamente un objeto con varios campos obligatorios y opcionales, evitando constructores largos y ambiguos, aplicando valores por defecto consistentes y centralizando la validación en build() antes de crear un objeto inmutable.

La decisión entre ambos —o ninguno— depende del tipo de variabilidad presente en el problema: variación de tipo concreto sugiere Factory Method; variación en la configuración de un mismo tipo sugiere Builder; ausencia de ambas sugiere que ningún patrón adicional está justificado.



12. Declaración de uso de IA

Para esta actividad utilicé herramientas de inteligencia artificial (Claude, Anthropic). La herramienta se empleó para generar el código base de Factory Method y Builder siguiendo la guía del curso, redactar el diagrama UML en PlantUML y estructurar el documento de análisis. Revisé, probé y adapté el contenido generado.